

AULA

API Mock

No contexto de desenvolvimento de software, uma API Mock (ou API simulada/falsa) é uma versão simulada de uma API real.

Em vez de se conectar a um servidor de verdade que processaria requisições e retornaria dados de um banco de dados, a API Mock intercepta as requisições e retorna dados predefinidos ou gerados artificialmente.

Pense nela como um dublê de cena para a API real. Ela se parece com a API original (endpoints, formatos de dados), mas não tem a complexidade ou a dependência de um backend real.

Comportamento Predeterminado: Você define exatamente quais respostas cada endpoint (rota) deve retornar para determinadas requisições (por exemplo, um GET em /users sempre retorna uma lista de 3 usuários fixos).

Independência de Backend: Não precisa de um servidor de banco de dados, lógica de negócio complexa ou autenticação real.

Para simular uma API, vamos usar o json-server.

1. Instalar a ferramenta dentro do seu projeto React:
`sudo npm install -g json-server` (precisa ser administrador)
2. Criar o arquivo de dados, `dados.json` ou `db.json` (ou qualquer outro)
3. Iniciar o json-server usando os dados criados:
`json-server --watch ./src/10-06-2025/dados.json`

O **json-server** irá atuar como um servidor local, disponibilizando o seu arquivo de dados como rotas.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

index.html      package-lock.json      public      src
sidarthacarvalho@MacBook-Air-de-Sidarta piw-02 % json-server --watch ./src/10-06-2025/dados.json
--watch/-w can be omitted, JSON Server 1+ watches for file changes by default
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching ./src/10-06-2025/dados.json...

♥( ͡° ͜ʖ ͡° )

Index:
http://localhost:3000/

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/characters
http://localhost:3000/episodes
```

Mock API

Nome da rota/endpoint

```
{
  "characters": [
    {
      "id": "spongebob",
      "name": "Bob Esponja Calça Quadrada"
    },
    {
      "id": "patrick",
      "name": "Patrick Estrela"
    }
  ],
  "episodes": [
    {
      "id": 1,
      "title": "Ajuda Procurada / Limpadores de Chaminé / Pescaria de Água-Viva"
    },
    {
      "id": 2,
      "title": "Fila da Felicidade / Biscoitos da Sorte"
    }
  ]
}
```

Arquivo de dados,
dados.json . Será
usado ao chamar o
json-server.

Mock API



```
[
  {
    "id": "spongebob",
    "name": "Bob Esponja Calça Quadrada",
    "species": "Esponja do Mar",
    "occupation": "Cozinheiro do Siri Cascudo",
    "main_location": "Abacaxi no Fundo do Biquíni",
    "friends": [
      "patrick",
      "sandy"
    ],
    "voice_actor": "Tom Kenny"
  },
  {
    "id": "patrick",
    "name": "Patrick Estrela",
    "species": "Estrela do Mar",
    "occupation": "Desempregado (ocasionalmente em 'empregos' estranhos)",
    "main_location": "Rocha no Fundo do Biquíni",
    "friends": [
      "spongebob"
    ],
    "voice_actor": "Bill Fagerbakke"
  },
  {
    "id": "squidward",
    "name": "Lula Molusco Tentáculos",
    "species": "Polvo",
    "occupation": "Caixa do Siri Cascudo / Artista / Músico",
    "main_location": "Moai (cabeça de pedra) no Fundo do Biquíni",
    "friends": [],
    "voice_actor": "Rodger Bumpass"
  },
  {
    "id": "sandy",
    "name": "Sandy Bochechas",
    "species": "Esquilo",
    "occupation": "Cientista / Inventor",
    "main_location": "Cúpula do Ar no Fundo do Biquíni",
    "friends": [
      "spongebob",
      "patrick"
    ],
    "voice_actor": "Carolyn Lawrence"
  }
],
```

A partir das rotas criadas, é só usar dentro do seu app React o `fetch()` para acessar os dados!

A partir do API Mock criado, vamos criar um componente React para ler as duas rotas que criamos e exibir os dados ao usuário.

Mock API

```
import {useState, useEffect} from "react";
import "../ListaDadosAPIMock.css";

export default function ListaDadosAPIMock() {
  const urlAPIMock = "http://localhost:3000/characters";
  const [personagens, setPersonagens] = useState([]);

  async function fetchDadosAPIMock() {
    let personagensRequest = await fetch(urlAPIMock);
    let personagensJson = await personagensRequest.json();
    setPersonagens(personagensJson);
  }

  useEffect(() => {
    fetchDadosAPIMock();
  }, []);

  return (
    <div>
      {personagens.map( (personagem) => {
        return <div className="card">
          <p>ID: {personagem.id}</p>
          <p>Nome: {personagem.name}</p>
          <p>Ocupação: {personagem.occupation}</p>
        </div>
      })}
    </div>
  )
}
```

ListaDadosAPIMock.jsx

Mock API

```
.card {  
  /* Dimensões e Espaçamento */  
  width: 300px; /* Largura fixa para o card */  
  padding: 20px; /* Espaçamento interno */  
  margin: 20px; /* Espaçamento externo para separar de outros elementos */  
  
  /* Estilo do Fundo e Borda */  
  background-color: #ccef5; /* Fundo branco */  
  border: 1px solid #e0e0e0; /* Borda cinza clara */  
  border-radius: 8px; /* Cantos arredondados */  
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1); /* Sombra suave para profundidade */  
  
  /* Alinhamento de Conteúdo (opcional, se quiser centralizar) */  
  text-align: center; /* Centraliza o texto e botões */  
  
  /* Transições para Interatividade (opcional, mas legal) */  
  transition: transform 0.3s ease, box-shadow 0.3s ease; /* Transição para hover */  
}  
/* Estilo para o hover (quando o mouse passa por cima) */  
.card:hover {  
  transform: translateY(-5px); /* Move o card ligeiramente para cima */  
  box-shadow: 0 8px 16px rgba(0, 0, 0, 0.2); /* Aumenta a sombra */  
}  
.card p {  
  color: #666666; /* Cor do parágrafo */  
  line-height: 1.6; /* Altura da linha para melhor leitura */  
  margin-bottom: 20px; /* Espaçamento abaixo do parágrafo */  
}
```

ListaDadosAPIMock.css

Mock API

ID: spongebob

Nome: Bob Esponja Calça Quadrada

Ocupação: Cozinheiro do Siri Cascudo

ID: patrick

Nome: Patrick Estrela

Ocupação: Desempregado
(ocasionalmente em 'empregos'
estranhos)

ID: squidward

Nome: Lula Molusco Tentáculos

Ocupação: Caixa do Siri Cascudo /
Artista / Músico